# Chapter 13: Native Functions and Modules

We have functions, closures, records, and a growing collection of operations such as `length`, `input`, and conversions. Several of those operations have their own parser and evaluator cases. Adding another useful operation that way means changing the interpreter again.

Two languages appear in this chapter. **Vertex** is the language our programs are written in. **Python** runs the interpreter, so it is the **host language**. Python code changes or configures the interpreter; Vertex code runs through it. The examples below are labeled with the language that executes them.

Suppose we want a square-root function. Python already has one. We need a way to connect a function value in Vertex to an implementation supplied by the host, without teaching the parser another special form:

**Vertex program**

```
sqrt = native("math.sqrt", ["x"]);
print(sqrt(9));
```

This prints `3.0`. The call to `native` creates a function value. The call to `sqrt` invokes the implementation connected to that value. Assignment, argument passing, and function calls all keep their existing meanings.

The second addition is a way to load a file as a program and keep the bindings it creates. A module is a program. Its global environment becomes the object returned by `import`. Together, these additions let a file provide functions written in Vertex, functions supplied by the host, and ordinary data through one interface.

## A Function Supplied by the Host

The interpreter is written in Python, so Python is the **host language** for this implementation. Vertex is the language whose programs it runs. Calling from an interpreted language into its host is one form of a **foreign function interface**. Our interface is deliberately small: a registered name, a parameter list, and an ordinary function call. (1)

**Vertex program**

```
sqrt = native("math.sqrt", ["x"]);
distance = native("math.hypot", ["a", "b"]);
print(sqrt(16));
print(distance(3, 4));
```

This prints `4.0`, then `5.0`. The host wrappers connect these names to Python’s square-root and Euclidean-distance operations. (2)

The strings `"x"`, `"a"`, and `"b"` name the Vertex parameters. They specify the function’s arity and must be distinct valid identifier names. The arguments themselves are passed positionally. The parameter spelling does not have to match the Python function’s parameter spelling.

`native` does not run the connected implementation when it creates the function. We can construct `sqrt`, store it, and never call it. Each construction produces a distinct function value; assigning that value to another variable keeps the same function identity.

## An Ordinary Function Value

Nothing about a function's use depends on whether its implementation is Vertex code or native code:

**Vertex program**

```
function apply(f, value) { return f(value) };
sqrt = native("math.sqrt", ["x"]);
operations = {"root": sqrt};
print(apply(operations.root, 25));
print(type(sqrt));
print(copy(sqrt) == sqrt);
```

This prints `5.0`, then `function`, then `true`. A native function can be stored in a record or list, passed to another function, returned, or retained by a closure. Like a Vertex function, it displays as `<function>`. Copying it preserves its identity. Its implementation fields are not accessible through Vertex indexing or field access.

Even `native` is an ordinary function value found through identifier lookup. We can give the constructor another name:

**Vertex program**

```
connect = native;
sqrt = connect("math.sqrt", ["value"]);
print(sqrt(36));
```

This prints `6.0`. No new keyword or parser production is needed. An ordinary local binding named `native` can shadow the supplied constructor, just as one ordinary binding can shadow another.

## Connecting Python Implementations

The **host registry** is a Python dictionary in `code/native_functions.py`. It connects a string name to a Python function. It belongs to the interpreter, not to a Vertex program's variables. Here is part of that dictionary:

**Python host**

```
NATIVE_FUNCTIONS = {
    "math.sqrt": sqrt,
    "math.hypot": hypot,
}
```

The key `"math.sqrt"` is a string. The value `sqrt` is a Python function already defined in that file. That wrapper checks Vertex's numeric rules, then calls Python's `math.sqrt`; for example, it rejects booleans instead of treating them as numbers. The dictionary also registers the file operations `io.read`, `io.write`, `io.readJSON`, and `io.writeJSON`.

Now return to the Vertex side:

**Vertex program**

```
root = native("math.sqrt", ["x"]);
print(root(25));
```

The constructor looks up `"math.sqrt"` in the Python dictionary and stores the resulting Python callable in a Vertex function value. It does not call the callable yet. `root(25)` later invokes it and gets `5.0` back.

| Step | Where it happens | What it does |
|---|---|---|
| Register `"math.sqrt"` | Python host | Store the Python function under that string key |
| `native("math.sqrt", ["x"])` | Vertex calls the interpreter | Look up the key and construct a function value |
| `root(25)` | Vertex calls the native function | Invoke the stored Python callable with `25` |

The dotted name does not import a Python module or evaluate Python source. It is simply a key we chose. An unregistered name fails even if Python has an operation with that spelling. Vertex cannot register an arbitrary Python function through this interface; registration happens in Python.

For a custom operation, the embedding Python program can register a callable:

**Python host**

```
from native_functions import register_native

def combine(a, b):
    return a + b

register_native("combine", combine)
```

After that setup, this **separate Vertex program** can obtain and call it:

**Vertex program**

```
f = native("combine", ["a", "b"]);
print(f(2, 5));
```

This prints `7`. Both registration and execution must use the same Python process, because the registry is an in-memory dictionary. Starting a new `runner.py` process would not retain a registration from another process. For example, the following Python code runs Vertex source after the setup above:

**Python host**

```
from evaluator import evaluate_source

# Python holds this text; evaluate_source runs its contents as Vertex.
vertex_source = '''
f = native("combine", ["a", "b"]);
print(f(2, 5));
'''
evaluate_source(vertex_source)
```

The outer code is Python. The text inside `vertex_source` is Vertex, and only `evaluate_source` interprets it. Re-registering a name later does not replace the callable already retained by an existing native function.

The registered name describes an interface rather than an implementation language. A Go implementation of the interpreter could connect the same name to a Go function. Keeping the same Vertex source useful across those hosts would require them to agree on argument types, results, and errors. We have implemented the Python connection here; there is no Go runtime in this chapter.

## Calls, Values, and Errors

The existing call handler does most of the work. It evaluates the callee, checks that it is a function, evaluates arguments left to right in the caller's environment, and checks the declared arity. For a Vertex function, it then builds a call frame and evaluates the body. For a native function, it invokes the connected Python callable with those evaluated arguments.

The host implementation must accept the declared number of positional arguments. A mismatched Vertex call fails its arity check before invoking the host. A mismatch between the declaration and the Python callable itself will instead raise Python's argument error when the host call occurs.

The result must be a Vertex value: null, a boolean, a number, a string, a function, a list, or an object with string keys. Nested values are checked. A wrapper returning a tuple or a Python datetime instance must first adapt that result to a supported language value. Lists and objects keep their identity, including aliases and cycles; the boundary does not silently copy them. A native operation can therefore mutate a list or object passed by its caller, just as a Vertex function can.

Unknown registrations raise `ValueError`. Bad argument declarations, invalid result values, and wrong arity raise the corresponding evaluator exceptions. Exceptions from Python implementations propagate to the host running the interpreter. They do not become null results. A returned tuple is not interpreted as a `(value, status)` pair: native operations return values, while evaluator control-flow statuses remain internal.

## A Module Is a Program

Put the following program in `modules/geometry.v`:

**Vertex program**

```
// file: modules/geometry.v
factor = 2;
function scale(x) { return factor * x };
function square(x) { return x * x };
sqrt = native("math.sqrt", ["x"]);
distance = native("math.hypot", ["a", "b"]);
```

There is no final `return` and no export declaration. Run that file through `import`:

**Vertex program**

```
geometry = import("modules/geometry.v");
print(geometry.factor);
print(geometry.square(5));
print(geometry.sqrt(9));
```

This prints `2`, then `25`, then `3.0`. Importing reads the file, tokenizes and parses it as a program, and evaluates it in a fresh global environment. When execution finishes normally, that environment is returned as an object. Each ordinary global binding becomes a field of that object.

The Vertex functions and the native functions are both callable values in the same result. A module can also contain lists, records, configuration values, or other imported modules. Dividing a larger program into such units is **modular programming**. (3)

`import` is another ordinary supplied function. It can be assigned to a name, passed as a value, or shadowed. Its single argument is a path string. The file has to be available to the interpreter process.

## The Object Is the Global Environment

The returned object is the module's actual global environment, not a snapshot of its bindings:

**Vertex program**

```
geometry = import("modules/geometry.v");
geometry.factor = 3;
print(geometry.scale(5));
```

This prints `15`. `scale` retains its defining environment through its closure. Changing the `factor` field changes that same environment's binding. Reads by the closure and writes through the module object meet at one place. Python's module objects similarly expose a namespace whose bindings are used by functions defined in the module, although Python's loading and caching rules are different from ours. (4)

| Reference | Points to |
| --- | --- |
| The importer's `geometry` binding | The module globals object |
| The module's `scale` field | A Vertex function value |
| The `scale` function's retained environment | That same module globals object |
| The module's `sqrt` field | A native function value |

We have deliberately made the environment and the public object refer to the same storage. That requires a distinction between environment lookup and field access. A field read such as `geometry.factor` still checks only that object. It does not search a parent environment. A module global has no caller environment as its lexical parent.

All ordinary globals are public for now, including helper functions and intermediate values. A leading underscore does not hide a field. Closures created inside a factory can still retain that factory's private local bindings, as before.

## Module State and Independent Imports

A module can maintain changing state with `set`:

**Vertex program**

```
// file: modules/counter.v
count = 0;
function next() {
    set count = count + 1;
    return count;
};
function reset() { set count = 0 };
```

Each import executes the program again in a new environment. There is no module cache in this version:

**Vertex program**

```
first = import("modules/counter.v");
second = import("modules/counter.v");
print(first.next());
print(first.next());
print(second.count);
```

This prints `1`, then `2`, then `0`. The two module objects have separate `count` bindings. Importing also repeats any top-level effects, including printing or other initialization work. Assigning `alias = first` instead would give two names for the same module instance.

Extracting a method preserves its connection to that module:

**Vertex program**

```
counter = import("modules/counter.v");
next = counter.next;
print(next());
print(counter.count);
```

This prints `1`, then `1`. The function does not need the module object passed back as a receiver. Its closure already retains the environment.

## Isolation and File Paths

The module starts with its own global environment. It can use supplied operations such as `native` and `import`, but it cannot read the importing program's local or global variables just because that program calls it. A missing module name raises the same unknown-identifier error as any other missing name.

A relative import path is resolved against the importing file's directory. A module in `modules/outer.v` that imports `"inner.v"` loads `modules/inner.v`. This rule also applies when an exported function imports a file later: the module remembers its source path. The command-line runner supplies the main program's path, so running it from another working directory does not change those relative imports. For source text passed directly to `evaluate_source`, the embedding host can supply `source_path`; otherwise relative paths start at the process's current working directory.

A missing file raises a file error. Invalid module syntax is a syntax error. Direct and indirect circular imports raise `ImportError`, with the resolved file chain in the message. The loader tracks imports in progress and clears that tracking even when evaluation fails. It does not cache a partially initialized module.

An imported file keeps ordinary program control-flow rules. A top-level `return`, `break`, or `continue` is still an error. `exit` propagates out of the import call and stops the enclosing program; the assignment receiving the module does not happen. An empty or comment-only file is now a valid program and returns an empty object when imported.

## Host Bookkeeping and Object Operations

The interpreter's `__input` and `__output` bookkeeping does not become module fields. The Python implementation uses a small dictionary subclass, `ModuleEnvironment`, with the source path and input/output state stored as host attributes. Ordinary module globals remain dictionary entries. Exported functions can print later without introducing an `__output` field into the module object.

The module's root environment also ends lexical lookup explicitly. A caller can store ordinary data under the string key `"$PARENT"`, but that does not give the module a new enclosing scope. Reserved environment links in ordinary call frames and a similarly spelled object field keep their different roles.

Module objects support the existing object operations: direct field and index access, structural equality, printing, and deep copying of data. Copying a module object does not clone its functions or retarget their closures:

**Vertex program**

```
geometry = import("modules/geometry.v");
other = copy(geometry);
other.factor = 9;
print(other.scale(5));
print(other.factor);
```

This prints `10`, then `9`. The copied record has its own `factor` field, but `scale` is the original function retaining the original module globals. Importing again, rather than copying, creates an independent module instance with independently created closures.

## A Native File Library

A module can collect related native functions behind one namespace. This small library supports text and JSON files, with matching operations in the Go host.

**Vertex program**

```
// file: modules/io.v
read = native("io.read", ["path"]);
write = native("io.write", ["path", "text"]);
readJSON = native("io.readJSON", ["path"]);
writeJSON = native("io.writeJSON", ["path", "value"]);
```

Text reads return a string. JSON reads return normal objects and lists, including nested scalar values. Writes create or replace the destination and return null.

**Vertex program**

```
io = import("modules/io.v");
io.write("note.txt", "hello");
text = io.read("note.txt");
io.writeJSON("record.json", {"name": "Ada", "scores": [1, 2, 3]});
data = io.readJSON("record.json");
print(text == "hello" and data.scores[2] == 3);
```

This prints `true`. Native file paths use the process working directory, while module import paths continue to use the importing source file's directory. Files are closed by the host wrappers. Errors propagate as normal Python exceptions. JSON writes check the data before opening the file: functions, cyclic data, and nonfinite numbers are rejected. Python uses its standard `json` module, adapting the language's values at the host boundary.

## Implementation

The native constructor creates a function value with the existing function identity marker, a copied parameter list, and a host callable. Native functions therefore reuse type, display, copy, and equality behavior. The call handler adds the host-call branch after its existing argument and arity checks. `native` itself is one of these function values.

`import` also enters through ordinary identifier lookup and call syntax. The loader needs the caller's source context and must propagate evaluator statuses, so the call handler recognizes this supplied function value and invokes its loader path. Aliasing `import` preserves that behavior; giving an unrelated function the same identifier spelling does not turn it into a loader. No new import keyword or declaration syntax is involved.

The loader constructs a fresh `ModuleEnvironment`, evaluates the parsed program there, and returns that very object on normal completion. Object operations recognize this dictionary subclass while function values keep their existing marker. Module globals can thus serve both as a lexical environment and as a public object without exposing host bookkeeping.

The earlier dedicated built-ins remain compatible in this implementation. The common native-function mechanism is available for new library functions; converting every older parser form to an ordinary function can be a separate refactoring. Modules need no special treatment for either kind of function.

## Verification

From this chapter's directory, run:

**Shell command**

```
python3 -m unittest discover -s code
python3 code/runner.py code/native.v
python3 code/runner.py code/modules.v
```

The native example prints `3.0`, `5.0`, `7.0`, `function`, and `true`. The module example prints `25`, `3.0`, `10`, `15`, `1`, `2`, and `0`. Tests cover the native interface, shared module bindings, independent imports, paths, cycles, errors, object operations, and the examples printed in this chapter. The earlier interpreter behavior remains covered by the inherited tests.

## Considerations

1. All ordinary module globals become public fields. When is that convenient, and which helper bindings might a larger library want to hide?
2. Why does returning the actual global environment make module field assignment agree with reads performed by exported closures?
3. What changes if repeated imports return a cached object instead of executing the file again? Consider both mutable state and initialization.
4. Which Python values need adapters before they can cross the native boundary? What should the interface contract say about mutation and errors?
5. Why does copying a module record leave its functions attached to the original module? How would a second import behave differently?

## Practice

1. Connect `math.hypot` under the name `distance` and use it to find the distance represented by the components `6` and `8`.
2. Write `applyTwice(f, value)`. Use a native square-root function to evaluate `applyTwice(sqrt, 81)`.
3. Import `modules/geometry.v`, change its `factor` to `4`, and predict `geometry.scale(3)`. Explain which environment supplies `factor`.
4. Import `modules/counter.v` twice. Increment the first instance twice, reset it, then increment the second. Print both `count` fields.
5. Write a Vertex module `modules/temperatures.v` with an `offset` of `32` and a function `toFahrenheit(c)` computing `c * 9 / 5 + offset`. Import it and convert `0` degrees Celsius.

## Answers to the Practice Questions

1. Something like:

   **Vertex program**

   ```
   distance = native("math.hypot", ["a", "b"]);
   print(distance(6, 8));
   ```

   This prints `10.0`.

2. Something like:

   **Vertex program**

   ```
   function applyTwice(f, value) { return f(f(value)) };
   sqrt = native("math.sqrt", ["x"]);
   print(applyTwice(sqrt, 81));
   ```

   This prints `3.0`.

3. Something like:

   **Vertex program**

   ```
   geometry = import("modules/geometry.v");
   geometry.factor = 4;
   print(geometry.scale(3));
   ```

   This prints `12`. The function's defining environment is the same object that `geometry` refers to, and its `factor` binding is now `4`.

4. Something like:

**Vertex program**

```
first = import("modules/counter.v");
second = import("modules/counter.v");
first.next();
first.next();
first.reset();
second.next();
print(first.count);
print(second.count);
```

This prints `0`, then `1`.

5. The module file contains:

   **Vertex program**

   ```
   // file: modules/temperatures.v
   offset = 32;
   function toFahrenheit(c) { return c * 9 / 5 + offset };
   ```

   The importing program is:

   **Vertex program**

   ```
   temperatures = import("modules/temperatures.v");
   print(temperatures.toFahrenheit(0));
   ```

   This prints `32.0`.

## References


1. Wikipedia contributors. “Foreign function interface.” Wikipedia. https://en.wikipedia.org/wiki/Foreign_function_interface
2. Python Software Foundation. “math - Mathematical functions,” entries for sqrt and hypot. Python documentation. https://docs.python.org/3/library/math.html
3. Wikipedia contributors. “Modular programming.” Wikipedia. https://en.wikipedia.org/wiki/Modular_programming
4. Python Software Foundation. “Data model,” section “Modules.” Python Language Reference. https://docs.python.org/3/reference/datamodel.html